Module 4: LAMAR

CONTENTS PAGE

01	code/historical/model_continual_pretrain/train.py	2
02	code/historical/model_1024_classifier/tokenize_data.py	10
03	code/historical/model_1024_classifier/train.py	11
04	code/historical/model_1024_classifier/evaluation_1024.py	24
05	code/historical/model_101_classifier/train_orca_ranking_101.py	30
06	code/historical/model_101_classifier/evaluation_top_ranking.py	41
07	code/historical/config_150M.json	50
08	code/historical/tokenizer/tokenizer_config.json	51
09	code/historical/tokenizer/special_tokens_map.json	52
10	code/historical/tokenizer/vocab.txt	53
11	scripts/validate_repository.py	54





code/historical/model_continual_pretrain/train.py

```python
import os

os.environ.setdefault("CUDA_VISIBLE_DEVICES", "0,1,2")
os.environ.setdefault("TOKENIZERS_PARALLELISM", "false")
os.environ.setdefault("PYTORCH_CUDA_ALLOC_CONF", "max_split_size_mb:128")
os.environ.setdefault("TORCH_DISTRIBUTED_DEBUG", "DETAIL")

import sys
import re
import argparse
import numpy as np
import torch

from datasets import load_dataset
from transformers import (
    AutoConfig,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    set_seed,
)

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT_DIR = os.path.abspath(os.path.join(SCRIPT_DIR, ".."))
if ROOT_DIR not in sys.path:
    sys.path.insert(0, ROOT_DIR)

LAMAR_ROOT = SCRIPT_DIR

from LAMAR.modeling_nucESM2 import EsmForMaskedLM

DEFAULT_TOKENIZER_PATH = os.path.join(ROOT_DIR, "tokenizer")
DEFAULT_MODEL_NAME = os.path.join(ROOT_DIR, "config_150M.json")

DEFAULT_PRETRAINED_MODEL_PATH = os.path.join(
    SCRIPT_DIR,
    "pretrained_base",
    "model.safetensors",
)

DEFAULT_TRAIN_PATH = os.path.join(
    ROOT_DIR,
    "data",
    "pretrain_dataset\human_transcriptome_pretraining_train.txt",
)

DEFAULT_VALID_PATH = os.path.join(
    ROOT_DIR,
    "data",
    "pretrain_dataset\human_transcriptome_pretraining_valid.txt",
)

DEFAULT_OUTPUT_DIR = os.path.join(
    SCRIPT_DIR,
    "output",
)
```





```

def normalize_seq(seq: str) -> str:
    seq = seq.strip().upper().replace("U", "T")
    seq = re.sub(r"[^ATCGN]", "N", seq)
    return seq


def load_pretrained_model(pretrained_model_path: str, config):
    load_dir = pretrained_model_path
    if os.path.isfile(pretrained_model_path):
        load_dir = os.path.dirname(pretrained_model_path)

    try:
        model = EsmForMaskedLM.from_pretrained(load_dir, config=config)
        return model
    except Exception as e:
        pass

    if not os.path.isfile(pretrained_model_path):
        raise FileNotFoundError(
            f"Pretrained model file not found: {pretrained_model_path}"
        )

    if not pretrained_model_path.endswith(".safetensors"):
        raise RuntimeError(
            "Fallback only supports .safetensors file. "
            "Please provide checkpoint dir or model.safetensors."
        )

    try:
        from safetensors.torch import load_file as load_safetensors
    except Exception as e:
        raise ImportError(
            "safetensors is required to load model.safetensors manually."
        ) from e

    model = EsmForMaskedLM(config)
    state_dict = load_safetensors(pretrained_model_path)
    missing_keys, unexpected_keys = model.load_state_dict(state_dict, strict=False)
    return model


def preprocess_logits_for_metrics(logits, labels):
    if isinstance(logits, tuple):
        logits = logits[0]
    return logits.argmax(dim=-1)


def compute_metrics(eval_pred):
    preds, labels = eval_pred
    mask = labels != -100
    if mask.sum() == 0:
        return {"masked_accuracy": 0.0}
    acc = (preds[mask] == labels[mask]).mean()
    return {"masked_accuracy": float(acc)}


class UnifiedMlmCollator:
    def __init__(self, tokenizer, mlm_probability=0.15):
        self.tokenizer = tokenizer
```





```
        self.mlm_probability = mlm_probability

    def _pad_labels(self, labels_list, max_len):
        padded = []
        for lab in labels_list:
            padded.append(lab + [-100] * (max_len - len(lab)))
        return torch.tensor(padded, dtype=torch.long)

    def _pad_attention_mask(self, attn_list, max_len):
        padded = []
        for am in attn_list:
            padded.append(am + [0] * (max_len - len(am)))
        return torch.tensor(padded, dtype=torch.long)

    def _pad_input_ids(self, ids_list, max_len):
        pad_id = self.tokenizer.pad_token_id
        padded = []
        for ids in ids_list:
            padded.append(ids + [pad_id] * (max_len - len(ids)))
        return torch.tensor(padded, dtype=torch.long)

    def _pad_special_tokens_mask(self, stm_list, max_len):
        padded = []
        for stm in stm_list:
            padded.append(stm + [1] * (max_len - len(stm)))
        return torch.tensor(padded, dtype=torch.long)

    def __call__(self, features):
        if "labels" in features[0]:
            input_ids = [f["input_ids"] for f in features]
            attention_mask = [
                f.get("attention_mask", [1] * len(f["input_ids"])) for f in features
            ]
            labels = [f["labels"] for f in features]

            max_len = max(len(x) for x in input_ids)

            batch = {
                "input_ids": self._pad_input_ids(input_ids, max_len),
                "attention_mask": self._pad_attention_mask(attention_mask, max_len),
                "labels": self._pad_labels(labels, max_len),
            }
            return batch

        input_ids = [f["input_ids"] for f in features]
        attention_mask = [
            f.get("attention_mask", [1] * len(f["input_ids"])) for f in features
        ]
        special_tokens_mask = [
            f.get("special_tokens_mask", [0] * len(f["input_ids"])) for f in features
        ]

        max_len = max(len(x) for x in input_ids)

        input_ids = self._pad_input_ids(input_ids, max_len)
        attention_mask = self._pad_attention_mask(attention_mask, max_len)
        special_tokens_mask = self._pad_special_tokens_mask(special_tokens_mask, max_len)

        labels = input_ids.clone()

        probability_matrix = torch.full(labels.shape, self.mlm_probability)
```





```
        probability_matrix.masked_fill_(special_tokens_mask.bool(), value=0.0)

        masked_indices = torch.bernoulli(probability_matrix).bool()
        labels[~masked_indices] = -100

        indices_replaced = (
            torch.bernoulli(torch.full(labels.shape, 0.8)).bool() & masked_indices
        )
        input_ids[indices_replaced] = self.tokenizer.mask_token_id

        indices_random = (
            torch.bernoulli(torch.full(labels.shape, 0.5)).bool()
            & masked_indices
            & ~indices_replaced
        )
        random_words = torch.randint(
            low=0,
            high=len(self.tokenizer),
            size=labels.shape,
            dtype=torch.long,
        )
        input_ids[indices_random] = random_words[indices_random]

        batch = {
            "input_ids": input_ids,
            "attention_mask": attention_mask,
            "labels": labels,
        }
        return batch


def tokenize_train_batch(examples, tokenizer, model_max_length):
    seqs = [normalize_seq(s) for s in examples["text"]]
    return tokenizer(
        seqs,
        truncation=True,
        max_length=model_max_length,
        return_special_tokens_mask=True,
    )


def tokenize_and_mask_eval_batch(
    examples,
    indices,
    tokenizer,
    model_max_length,
    mlm_probability,
    seed,
):
    seqs = [normalize_seq(s) for s in examples["text"]]
    encodings = tokenizer(
        seqs,
        truncation=True,
        max_length=model_max_length,
        return_special_tokens_mask=True,
    )

    special_ids = set(tokenizer.all_special_ids)
    candidate_random_token_ids = [
        i for i in range(len(tokenizer)) if i not in special_ids
    ]
```





```
    if len(candidate_random_token_ids) == 0:
        candidate_random_token_ids = [tokenizer.mask_token_id]

    out_input_ids = []
    out_attention_mask = []
    out_labels = []

    for i, input_ids in enumerate(encodings["input_ids"]):
        special_tokens_mask = encodings["special_tokens_mask"][i]
        idx = indices[i]
        rng = np.random.RandomState(seed + int(idx))

        labels = [-100] * len(input_ids)

        for pos, is_special in enumerate(special_tokens_mask):
            if is_special:
                continue

            if rng.rand() < mlm_probability:
                labels[pos] = input_ids[pos]
                prob = rng.rand()
                if prob < 0.8:
                    input_ids[pos] = tokenizer.mask_token_id
                elif prob < 0.9:
                    input_ids[pos] = int(
                        candidate_random_token_ids[
                            rng.randint(len(candidate_random_token_ids))
                        ]
                    )
                else:
                    pass

        out_input_ids.append(input_ids)
        out_attention_mask.append([1] * len(input_ids))
        out_labels.append(labels)

    return {
        "input_ids": out_input_ids,
        "attention_mask": out_attention_mask,
        "labels": out_labels,
    }


def main():
    parser = argparse.ArgumentParser(
        description="Continue pretraining LAMAR on human 3'UTR"
    )

    parser.add_argument("--tokenizer_path", type=str, default=DEFAULT_TOKENIZER_PATH)
    parser.add_argument("--model_name", type=str, default=DEFAULT_MODEL_NAME)
    parser.add_argument(
        "--pretrained_model_path", type=str, default=DEFAULT_PRETRAINED_MODEL_PATH
    )
    parser.add_argument("--data_for_pretrain_path", type=str, default=DEFAULT_TRAIN_PATH)
    parser.add_argument("--validation_data_path", type=str, default=DEFAULT_VALID_PATH)
    parser.add_argument("--output_dir", type=str, default=DEFAULT_OUTPUT_DIR)

    parser.add_argument("--model_max_length", type=int, default=1026)
    parser.add_argument("--token_dropout", action="store_true")
    parser.add_argument("--positional_embedding_type", type=str, default="rotary")
    parser.add_argument("--hidden_size", type=int, default=768)
```





```
    parser.add_argument("--intermediate_size", type=int, default=3072)
    parser.add_argument("--num_attention_heads", type=int, default=12)
    parser.add_argument("--num_hidden_layers", type=int, default=12)

    parser.add_argument("--batch_size", type=int, default=8)
    parser.add_argument("--eval_batch_size", type=int, default=4)
    parser.add_argument("--peak_lr", type=float, default=1e-4)
    parser.add_argument("--warmup_ratio", type=float, default=0.02)
    parser.add_argument("--max_steps", type=int, default=100000)
    parser.add_argument("--grad_clipping_norm", type=float, default=1.0)
    parser.add_argument("--accum_steps", type=int, default=1)
    parser.add_argument("--save_steps", type=int, default=1000)
    parser.add_argument("--eval_steps", type=int, default=1000)
    parser.add_argument("--logging_steps", type=int, default=100)
    parser.add_argument("--max_eval_samples", type=int, default=10000)

    parser.add_argument("--fp16", action="store_true", default=True)
    parser.add_argument("--disable_tqdm", action="store_true")
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--mlm_probability", type=float, default=0.15)

    args = parser.parse_args()

    os.makedirs(args.output_dir, exist_ok=True)
    set_seed(args.seed)

    if "LOCAL_RANK" not in os.environ:
        print(
            "[WARN] Not launched via torchrun. "
            "To use all 3 GPUs (0,1,2) in parallel, run: "
            "torchrun --nproc_per_node 3 pretrain_continue.py"
        )

    tokenizer = AutoTokenizer.from_pretrained(
        args.tokenizer_path,
        model_max_length=args.model_max_length,
        use_fast=True,
    )

    if tokenizer.pad_token is None:
        if tokenizer.eos_token is not None:
            tokenizer.pad_token = tokenizer.eos_token
        elif tokenizer.sep_token is not None:
            tokenizer.pad_token = tokenizer.sep_token
        else:
            raise ValueError(
                "Tokenizer has no pad/eos/sep token. "
                "Please check tokenizer/single_nucleotide."
            )

    config = AutoConfig.from_pretrained(
        args.model_name,
        vocab_size=len(tokenizer),
        pad_token_id=tokenizer.pad_token_id,
        mask_token_id=tokenizer.mask_token_id,
        token_dropout=args.token_dropout,
        positional_embedding_type=args.positional_embedding_type,
        hidden_size=args.hidden_size,
        intermediate_size=args.intermediate_size,
        num_attention_heads=args.num_attention_heads,
        num_hidden_layers=args.num_hidden_layers,
```





```
    )

    if hasattr(config, "max_position_embeddings"):
        config.max_position_embeddings = args.model_max_length

    raw_datasets = load_dataset(
        "text",
        data_files={
            "train": args.data_for_pretrain_path,
            "validation": args.validation_data_path,
        },
    )

    raw_datasets = raw_datasets.filter(lambda x: len(x["text"].strip()) >= 50)

    if (
        args.max_eval_samples is not None
        and len(raw_datasets["validation"]) > args.max_eval_samples
    ):
        raw_datasets["validation"] = (
            raw_datasets["validation"]
            .shuffle(seed=args.seed)
            .select(range(args.max_eval_samples))
        )

    def train_tokenize_fn(examples):
        return tokenize_train_batch(examples, tokenizer, args.model_max_length)

    train_dataset = raw_datasets["train"].map(
        train_tokenize_fn,
        batched=True,
        remove_columns=["text"],
        desc="Tokenizing train dataset",
    )

    def valid_tokenize_mask_fn(examples, indices):
        return tokenize_and_mask_eval_batch(
            examples=examples,
            indices=indices,
            tokenizer=tokenizer,
            model_max_length=args.model_max_length,
            mlm_probability=args.mlm_probability,
            seed=args.seed,
        )

    eval_dataset = raw_datasets["validation"].map(
        valid_tokenize_mask_fn,
        batched=True,
        with_indices=True,
        remove_columns=["text"],
        desc="Tokenizing and masking validation dataset",
    )

    collator = UnifiedMlmCollator(
        tokenizer=tokenizer,
        mlm_probability=args.mlm_probability,
    )

    model = load_pretrained_model(args.pretrained_model_path, config)

    try:
```





```
        from LAMAR.flash_attn_patch import EsmSelfAttentionAddFlashAttnPatch

        for i in range(config.num_hidden_layers):
            model.esm.encoder.layer[i].attention.self = EsmSelfAttentionAddFlashAttnPatch(
                config, position_embedding_type=args.positional_embedding_type
            )
    except Exception as e:
        pass

    train_args = TrainingArguments(
        output_dir=args.output_dir,
        overwrite_output_dir=True,
        per_device_train_batch_size=args.batch_size,
        per_device_eval_batch_size=args.eval_batch_size,
        gradient_accumulation_steps=args.accum_steps,
        learning_rate=args.peak_lr,
        weight_decay=0.01,
        adam_beta1=0.9,
        adam_beta2=0.98,
        adam_epsilon=1e-8,
        warmup_ratio=args.warmup_ratio,
        max_steps=args.max_steps,
        max_grad_norm=args.grad_clipping_norm,
        evaluation_strategy="steps",
        eval_steps=args.eval_steps,
        logging_steps=args.logging_steps,
        save_strategy="steps",
        save_steps=args.save_steps,
        save_total_limit=3,
        load_best_model_at_end=True,
        metric_for_best_model="masked_accuracy",
        greater_is_better=True,
        dataloader_drop_last=True,
        fp16=args.fp16,
        report_to="none",
        disable_tqdm=args.disable_tqdm,
        save_safetensors=True,
        remove_unused_columns=False,
        ddp_find_unused_parameters=True,
    )

    trainer = Trainer(
        model=model,
        args=train_args,
        train_dataset=train_dataset,
        eval_dataset=eval_dataset,
        data_collator=collator,
        tokenizer=tokenizer,
        compute_metrics=compute_metrics,
        preprocess_logits_for_metrics=preprocess_logits_for_metrics,
    )

    trainer.train()

    trainer.save_model(args.output_dir)
    tokenizer.save_pretrained(args.output_dir)


if __name__ == "__main__":
    main()
```





code/historical/model_1024_classifier/tokenize_data.py

```python
# -*- coding: utf-8 -*-
from transformers import AutoTokenizer
from datasets import load_dataset
import multiprocessing as mp
import os

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT_DIR = os.path.abspath(os.path.join(SCRIPT_DIR, ".."))

tokenizer = AutoTokenizer.from_pretrained(
    os.path.join(ROOT_DIR, "tokenizer"), model_max_length=103
)


def group_texts(examples):
    tokenized_inputs = tokenizer(
        examples["seq"], truncation=True, max_length=tokenizer.model_max_length
    )
    tokenized_inputs["labels"] = [int(x) for x in examples["label"]]
    return tokenized_inputs


train_set_path = os.path.join(ROOT_DIR, "data", "1024nt_dataset", "orca_puf12_train.csv")
valid_set_path = os.path.join(ROOT_DIR, "data", "1024nt_dataset", "orca_puf12_valid.csv")

data_path = os.path.join(ROOT_DIR, "data", "seq_1024_orca_puf12")

dataset = load_dataset(
    "csv", data_files={'train': train_set_path, 'valid': valid_set_path}
)

data = dataset.map(group_texts, batched=True, num_proc=mp.cpu_count())

data.save_to_disk(data_path, num_proc=mp.cpu_count())

print(data)
```





code/historical/model_1024_classifier/train.py

```python
#!/usr/bin/env python3

import os
import sys

os.environ.setdefault("TOKENIZERS_PARALLELISM", "false")

import gc
import json
import math
import random
import shutil
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import optuna

from datasets import load_from_disk, Dataset, DatasetDict

from transformers import (
    AutoConfig,
    AutoTokenizer,
    DataCollatorWithPadding,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback,
    TrainerCallback,
    set_seed as hf_set_seed,
)

from safetensors.torch import load_file

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    confusion_matrix,
)

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT_DIR = os.path.abspath(os.path.join(SCRIPT_DIR, ".."))

LAMAR_ROOT = SCRIPT_DIR

if LAMAR_ROOT not in sys.path:
    sys.path.insert(0, LAMAR_ROOT)

from LAMAR.sequence_classification_patch import EsmForSequenceClassification


TRAIN_PATH = os.path.join(ROOT_DIR, "data", "seq_1024_orca_puf12", "train")
```





```
VALIDATION_PATH = os.path.join(ROOT_DIR, "data", "seq_1024_orca_puf12", "valid")
PRETRAIN_STATE_PATH = os.path.join(
    ROOT_DIR,
    "model_continual_pretrain",
    "output",
    "transcriptome_pretrain",
    "checkpoint-100000",
    "model.safetensors",
)
OUTPUT_ROOT = os.path.join(SCRIPT_DIR, "output")
TOKENIZER_PATH = os.path.join(ROOT_DIR, "tokenizer")
MODEL_CONFIG_PATH = os.path.join(ROOT_DIR, "config_150M.json")


MODEL_MAX_LENGTH = 1026
POSITIONAL_EMBEDDING_TYPE = "rotary"
HIDDEN_SIZE = 768
INTERMEDIATE_SIZE = 3072
NUM_ATTENTION_HEADS = 12
NUM_HIDDEN_LAYERS = 12
NUM_LABELS = 2
HEAD_TYPE = "Linear"
FREEZE_ENCODER = False
KERNEL_SIZES = [2, 3, 5]
OCS = 32
FLASH_ATTENTION = False

RANDOM_SEED = 42
N_TRIALS = 24
MAX_EPOCHS = 15
EARLY_STOPPING_PATIENCE = 10
BEST_METRIC = "pr_auc"

BATCH_CONFIGS = {
    "batch4_accum2": {
        "per_device_train_batch_size": 4,
        "gradient_accumulation_steps": 2,
    },
    "batch8_accum1": {
        "per_device_train_batch_size": 8,
        "gradient_accumulation_steps": 1,
    },
    "batch8_accum2": {
        "per_device_train_batch_size": 8,
        "gradient_accumulation_steps": 2,
    },
    "batch16_accum1": {
        "per_device_train_batch_size": 16,
        "gradient_accumulation_steps": 1,
    },
}


Path(OUTPUT_ROOT).mkdir(parents=True, exist_ok=True)


def set_all_seeds(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```





```
    hf_set_seed(seed)


set_all_seeds(RANDOM_SEED)


def check_paths():
    required_paths = {
        "TRAIN_PATH": TRAIN_PATH,
        "VALIDATION_PATH": VALIDATION_PATH,
        "PRETRAIN_STATE_PATH": PRETRAIN_STATE_PATH,
        "TOKENIZER_PATH": TOKENIZER_PATH,
        "MODEL_CONFIG_PATH": MODEL_CONFIG_PATH,
    }
    for name, path in required_paths.items():
        if not os.path.exists(path):
            raise FileNotFoundError(f"{name} does not exist: {path}")
    print("All required paths exist.")


check_paths()

print("CUDA available:", torch.cuda.is_available())
print("Visible GPU count:", torch.cuda.device_count())

if torch.cuda.is_available():
    for gpu_index in range(torch.cuda.device_count()):
        print(f"GPU {gpu_index}: {torch.cuda.get_device_name(gpu_index)}")


print("\nLoading tokenizer...")

tokenizer = AutoTokenizer.from_pretrained(
    TOKENIZER_PATH,
    model_max_length=MODEL_MAX_LENGTH,
)

if tokenizer.pad_token_id is None:
    if tokenizer.eos_token is not None:
        tokenizer.pad_token = tokenizer.eos_token
    else:
        raise ValueError("Tokenizer has no pad_token and no eos_token")

print("Tokenizer vocabulary size:", len(tokenizer))
print("Tokenizer pad_token_id:", tokenizer.pad_token_id)
print("Tokenizer model_max_length:", tokenizer.model_max_length)

data_collator = DataCollatorWithPadding(
    tokenizer=tokenizer,
    padding=True,
    return_tensors="pt",
)


def unwrap_dataset(dataset_object, preferred_split):
    if isinstance(dataset_object, Dataset):
        return dataset_object

    if isinstance(dataset_object, DatasetDict):
        if preferred_split in dataset_object:
            return dataset_object[preferred_split]
```





```
        available_splits = list(dataset_object.keys())
        if not available_splits:
            raise ValueError(f"No dataset split found for {preferred_split}")
        print(
            f"Warning: split '{preferred_split}' not found. Using '{available_splits[0]}' instead.
"
        )
        return dataset_object[available_splits[0]]

    raise TypeError(f"Unsupported dataset object type: {type(dataset_object)}")


print("\nLoading training dataset...")
train_object = load_from_disk(TRAIN_PATH)
train_dataset = unwrap_dataset(train_object, "train")

print("\nLoading validation dataset...")
validation_object = load_from_disk(VALIDATION_PATH)
validation_dataset = unwrap_dataset(validation_object, "valid")

print("\nDataset summary:")
print("Train size:", len(train_dataset))
print("Validation size:", len(validation_dataset))
print("Train columns:", train_dataset.column_names)
print("Validation columns:", validation_dataset.column_names)


def normalize_label_column(dataset, dataset_name):
    columns = dataset.column_names
    if "labels" in columns:
        return dataset
    if "label" in columns:
        print(f"Renaming '{dataset_name}.label' to '{dataset_name}.labels'")
        return dataset.rename_column("label", "labels")
    raise ValueError(
        f"{dataset_name} must contain 'labels' or 'label'. Available columns: {columns}"
    )


train_dataset = normalize_label_column(train_dataset, "train")
validation_dataset = normalize_label_column(validation_dataset, "validation")


def print_label_distribution(dataset, dataset_name):
    labels = np.asarray(dataset["labels"], dtype=np.int64)
    unique, counts = np.unique(labels, return_counts=True)
    distribution = {int(label): int(count) for label, count in zip(unique, counts)}
    print(f"{dataset_name} label distribution:", distribution)
    if len(unique) < 2:
        raise ValueError(f"{dataset_name} contains only one class.")


print_label_distribution(train_dataset, "Train")
print_label_distribution(validation_dataset, "Validation")


def softmax_numpy(logits):
    logits = np.asarray(logits)
    logits = logits - np.max(logits, axis=1, keepdims=True)
    exp_logits = np.exp(logits)
    return exp_logits / np.sum(exp_logits, axis=1, keepdims=True)
```





```
239
240
241 def compute_metrics(eval_prediction):
242     predictions = eval_prediction.predictions
243     labels = eval_prediction.label_ids
244
245     if isinstance(predictions, tuple):
246         predictions = predictions[0]
247
248     predictions = np.asarray(predictions)
249     labels = np.asarray(labels).reshape(-1)
250
251     probabilities = softmax_numpy(predictions)
252     positive_probabilities = probabilities[:, 1]
253     predicted_classes = np.argmax(predictions, axis=1)
254
255     metrics = {
256         "accuracy": float(accuracy_score(labels, predicted_classes)),
257         "precision": float(precision_score(labels, predicted_classes, zero_division=0)),
258         "recall": float(recall_score(labels, predicted_classes, zero_division=0)),
259         "f1": float(f1_score(labels, predicted_classes, zero_division=0)),
260     }
261
262     try:
263         metrics["roc_auc"] = float(roc_auc_score(labels, positive_probabilities))
264     except Exception:
265         metrics["roc_auc"] = float("nan")
266
267     try:
268         metrics["pr_auc"] = float(average_precision_score(labels, positive_probabilities))
269     except Exception:
270         metrics["pr_auc"] = float("nan")
271
272     tn, fp, fn, tp = confusion_matrix(labels, predicted_classes, labels=[0, 1]).ravel()
273
274     metrics["true_negative"] = int(tn)
275     metrics["false_positive"] = int(fp)
276     metrics["false_negative"] = int(fn)
277     metrics["true_positive"] = int(tp)
278
279     return metrics
280
281
282 def build_model(token_dropout_value):
283     config_kwargs = dict(
284         vocab_size=len(tokenizer),
285         pad_token_id=tokenizer.pad_token_id,
286         num_labels=NUM_LABELS,
287         token_dropout=token_dropout_value,
288         positional_embedding_type=POSITIONAL_EMBEDDING_TYPE,
289         hidden_size=HIDDEN_SIZE,
290         intermediate_size=INTERMEDIATE_SIZE,
291         num_attention_heads=NUM_ATTENTION_HEADS,
292         num_hidden_layers=NUM_HIDDEN_LAYERS,
293     )
294
295     mask_token_id = getattr(tokenizer, "mask_token_id", None)
296     if mask_token_id is not None:
297         config_kwargs["mask_token_id"] = mask_token_id
298
299     config = AutoConfig.from_pretrained(MODEL_CONFIG_PATH, **config_kwargs)
```





```
300
301     print(f"Creating model. token_dropout={token_dropout_value}")
302
303     model = EsmForSequenceClassification(
304         config,
305         head_type=HEAD_TYPE,
306         freeze=FREEZE_ENCODER,
307         kernel_sizes=KERNEL_SIZES,
308         ocs=OCS,
309     )
310
311     if FLASH_ATTENTION:
312         from flash_attn_patch import EsmSelfAttentionAddFlashAttnPatch
313
314         for layer_index in range(config.num_hidden_layers):
315             model.esm.encoder.layer[layer_index].attention.self = (
316                 EsmSelfAttentionAddFlashAttnPatch(
317                     config,
318                     position_embedding_type="rotary",
319                 )
320             )
321
322     print("Loading pretrained weights:", PRETRAIN_STATE_PATH)
323
324     state_dict = load_file(PRETRAIN_STATE_PATH, device="cpu")
325     missing_keys, unexpected_keys = model.load_state_dict(state_dict, strict=False)
326
327     print("Pretrained missing keys:", len(missing_keys))
328     print("Pretrained unexpected keys:", len(unexpected_keys))
329
330     if len(missing_keys) > 30:
331         print(
332             "Warning: many parameters were not loaded. Please verify model architecture compatibil
  .
ity."
333         )
334
335     return model
336
337
338 class OptunaPruningCallback(TrainerCallback):
339     def __init__(self, trial):
340         self.trial = trial
341         self.report_step = 0
342
343     def on_evaluate(self, args, state, control, metrics=None, **kwargs):
344         if metrics is None:
345             return control
346
347         current_value = metrics.get(f"eval_{BEST_METRIC}", None)
348         if current_value is None:
349             return control
350
351         if not np.isfinite(current_value):
352             current_value = 0.0
353
354         self.report_step += 1
355
356         self.trial.report(float(current_value), step=self.report_step)
357
358         if self.trial.should_prune():
359             print(
```





```
                f"Trial {self.trial.number} pruned at "
                f"report_step {self.report_step}, "
                f"eval_{BEST_METRIC}={current_value:.6f}"
            )
            raise optuna.TrialPruned()

        return control


def sample_hyperparameters(trial):
    batch_config_name = trial.suggest_categorical(
        "batch_config",
        list(BATCH_CONFIGS.keys()),
    )
    batch_config = BATCH_CONFIGS[batch_config_name]

    params = {
        "learning_rate": trial.suggest_float("learning_rate", 1e-5, 1e-4, log=True),
        "weight_decay": trial.suggest_float("weight_decay", 1e-6, 1e-2, log=True),
        "token_dropout": trial.suggest_float("token_dropout", 0.0, 0.05),
        "warmup_ratio": trial.suggest_float("warmup_ratio", 0.0, 0.1),
        "batch_config": batch_config_name,
        "per_device_train_batch_size": batch_config["per_device_train_batch_size"],
        "gradient_accumulation_steps": batch_config["gradient_accumulation_steps"],
        "num_train_epochs": MAX_EPOCHS,
    }

    return params


def get_visible_gpu_count():
    if torch.cuda.is_available():
        return max(1, torch.cuda.device_count())
    return 1


def compute_eval_steps(train_size, effective_batch_size):
    updates_per_epoch = max(1, math.ceil(train_size / effective_batch_size))
    eval_steps = max(10, updates_per_epoch // 2)
    return updates_per_epoch, eval_steps


def build_training_args(output_dir, params):
    gpu_count = get_visible_gpu_count()
    per_device_batch_size = params["per_device_train_batch_size"]
    accumulation_steps = params["gradient_accumulation_steps"]
    effective_batch_size = per_device_batch_size * gpu_count * accumulation_steps

    updates_per_epoch, eval_steps = compute_eval_steps(
        len(train_dataset),
        effective_batch_size,
    )

    print("GPU count:", gpu_count)
    print("Per-device batch size:", per_device_batch_size)
    print("Gradient accumulation steps:", accumulation_steps)
    print("Estimated effective batch size:", effective_batch_size)
    print("Estimated updates per epoch:", updates_per_epoch)
    print("Evaluation steps:", eval_steps)

    return TrainingArguments(
```





```
        output_dir=str(output_dir),
        per_device_train_batch_size=per_device_batch_size,
        per_device_eval_batch_size=per_device_batch_size,
        gradient_accumulation_steps=accumulation_steps,
        learning_rate=params["learning_rate"],
        weight_decay=params["weight_decay"],
        warmup_ratio=params["warmup_ratio"],
        num_train_epochs=params["num_train_epochs"],
        adam_beta1=0.9,
        adam_beta2=0.98,
        adam_epsilon=1e-8,
        max_grad_norm=1.0,
        dataloader_drop_last=False,
        evaluation_strategy="steps",
        eval_steps=eval_steps,
        save_strategy="steps",
        save_steps=eval_steps,
        save_total_limit=3,
        logging_strategy="steps",
        logging_steps=max(1, eval_steps // 2),
        load_best_model_at_end=True,
        metric_for_best_model=BEST_METRIC,
        greater_is_better=True,
        fp16=False,
        bf16=False,
        report_to="none",
        save_safetensors=True,
        remove_unused_columns=True,
        seed=RANDOM_SEED,
        data_seed=RANDOM_SEED,
    )


def run_trial(trial):
    params = sample_hyperparameters(trial)

    trial_name = f"trial_{trial.number:04d}"
    trial_output_dir = Path(OUTPUT_ROOT) / "optuna_trials" / trial_name

    if trial_output_dir.exists():
        shutil.rmtree(trial_output_dir)
    trial_output_dir.mkdir(parents=True, exist_ok=True)

    print("\n" + "=" * 80)
    print(f"Starting Optuna trial {trial.number}")
    print(json.dumps(params, indent=2))
    print("=" * 80)

    set_all_seeds(RANDOM_SEED + trial.number)

    model = build_model(token_dropout_value=params["token_dropout"])
    training_args = build_training_args(trial_output_dir, params)

    callbacks = [
        EarlyStoppingCallback(early_stopping_patience=EARLY_STOPPING_PATIENCE),
        OptunaPruningCallback(trial),
    ]

    trainer = Trainer(
        model=model,
        args=training_args,
```





```
        train_dataset=train_dataset,
        eval_dataset=validation_dataset,
        compute_metrics=compute_metrics,
        data_collator=data_collator,
        tokenizer=tokenizer,
        callbacks=callbacks,
    )

    try:
        trainer.train()

        metrics = trainer.evaluate(eval_dataset=validation_dataset)

        objective_value = metrics.get(f"eval_{BEST_METRIC}", 0.0)
        if not np.isfinite(objective_value):
            objective_value = 0.0

        with open(trial_output_dir / "trial_params.json", "w") as handle:
            json.dump(params, handle, indent=2)

        with open(trial_output_dir / "validation_metrics.json", "w") as handle:
            json.dump(metrics, handle, indent=2)

        log_history = trainer.state.log_history
        with open(trial_output_dir / "log_history.json", "w") as handle:
            json.dump(log_history, handle, indent=2)

        print(
            f"Trial {trial.number} finished. "
            f"Validation {BEST_METRIC.upper()}={objective_value:.6f}"
        )

        return float(objective_value)

    except optuna.TrialPruned:
        print(f"Trial {trial.number} was pruned.")
        raise

    finally:
        del trainer
        del model

        gc.collect()

        if torch.cuda.is_available():
            torch.cuda.empty_cache()


def convert_best_params(best_trial):
    params = dict(best_trial.params)

    batch_config_name = params["batch_config"]
    batch_config = BATCH_CONFIGS[batch_config_name]

    params["per_device_train_batch_size"] = batch_config["per_device_train_batch_size"]
    params["gradient_accumulation_steps"] = batch_config["gradient_accumulation_steps"]
    params["num_train_epochs"] = MAX_EPOCHS

    return params


```





```python
def train_final_model(best_trial):
    best_params = convert_best_params(best_trial)

    print("\n" + "=" * 80)
    print("Training final model")
    print("=" * 80)

    print(f"Best validation {BEST_METRIC.upper()} from search:", best_trial.value)
    print("Final training parameters:")
    print(json.dumps(best_params, indent=2))

    final_output_dir = Path(OUTPUT_ROOT) / "final_model"

    if final_output_dir.exists():
        shutil.rmtree(final_output_dir)
    final_output_dir.mkdir(parents=True, exist_ok=True)

    set_all_seeds(RANDOM_SEED + 10000)

    model = build_model(token_dropout_value=best_params["token_dropout"])

    per_device_batch_size = best_params["per_device_train_batch_size"]
    accumulation_steps = best_params["gradient_accumulation_steps"]
    gpu_count = get_visible_gpu_count()
    effective_batch_size = per_device_batch_size * accumulation_steps * gpu_count

    updates_per_epoch, eval_steps = compute_eval_steps(
        len(train_dataset),
        effective_batch_size,
    )

    final_args = TrainingArguments(
        output_dir=str(final_output_dir),
        per_device_train_batch_size=per_device_batch_size,
        per_device_eval_batch_size=per_device_batch_size,
        gradient_accumulation_steps=accumulation_steps,
        learning_rate=best_params["learning_rate"],
        weight_decay=best_params["weight_decay"],
        warmup_ratio=best_params["warmup_ratio"],
        num_train_epochs=MAX_EPOCHS,
        adam_beta1=0.9,
        adam_beta2=0.98,
        adam_epsilon=1e-8,
        max_grad_norm=1.0,
        dataloader_drop_last=False,
        evaluation_strategy="steps",
        eval_steps=eval_steps,
        save_strategy="steps",
        save_steps=eval_steps,
        save_total_limit=3,
        logging_strategy="steps",
        logging_steps=max(1, eval_steps // 2),
        load_best_model_at_end=True,
        metric_for_best_model=BEST_METRIC,
        greater_is_better=True,
        fp16=False,
        bf16=False,
        report_to="none",
        save_safetensors=True,
        remove_unused_columns=True,
        seed=RANDOM_SEED + 10000,
```





```
        data_seed=RANDOM_SEED + 10000,
    )

    trainer = Trainer(
        model=model,
        args=final_args,
        train_dataset=train_dataset,
        eval_dataset=validation_dataset,
        compute_metrics=compute_metrics,
        data_collator=data_collator,
        tokenizer=tokenizer,
        callbacks=[
            EarlyStoppingCallback(early_stopping_patience=EARLY_STOPPING_PATIENCE),
        ],
    )

    print("Final model effective batch size:", effective_batch_size)
    print("Final model updates per epoch:", updates_per_epoch)
    print("Final model evaluation steps:", eval_steps)

    trainer.train()

    print("\nEvaluating final model on validation dataset...")
    validation_metrics = trainer.evaluate(eval_dataset=validation_dataset)

    with open(final_output_dir / "best_hyperparameters.json", "w") as handle:
        json.dump(best_params, handle, indent=2)

    with open(final_output_dir / "validation_metrics.json", "w") as handle:
        json.dump(validation_metrics, handle, indent=2)

    with open(final_output_dir / "log_history.json", "w") as handle:
        json.dump(trainer.state.log_history, handle, indent=2)

    print("\nFinal validation metrics:")
    print(json.dumps(validation_metrics, indent=2))

    print("Best checkpoint:", trainer.state.best_model_checkpoint)
    print(f"Best validation {BEST_METRIC.upper()}:", trainer.state.best_metric)

    return validation_metrics


def main():
    study_db_path = Path(OUTPUT_ROOT) / "optuna_seq1026.db"
    study_database = "sqlite:///" + str(study_db_path)

    print("\nOptuna database:", study_database)

    study = optuna.create_study(
        study_name="lamar_seq1026_classification",
        storage=study_database,
        load_if_exists=True,
        direction="maximize",
        sampler=optuna.samplers.TPESampler(seed=RANDOM_SEED),
        pruner=optuna.pruners.HyperbandPruner(
            min_resource=1,
            max_resource=MAX_EPOCHS * 2,
            reduction_factor=3,
        ),
    )
```





```
completed_or_pruned = len(
    [
        trial
        for trial in study.trials
        if trial.state
        in (
            optuna.trial.TrialState.COMPLETE,
            optuna.trial.TrialState.PRUNED,
        )
    ]
)

remaining_trials = max(0, N_TRIALS - completed_or_pruned)

print("Existing completed/pruned trials:", completed_or_pruned)
print("Remaining trials:", remaining_trials)

if remaining_trials > 0:
    study.optimize(
        run_trial,
        n_trials=remaining_trials,
        gc_after_trial=True,
    )

completed_trials = [
    trial for trial in study.trials if trial.state == optuna.trial.TrialState.COMPLETE
]

if not completed_trials:
    raise RuntimeError("No completed Optuna trial found.")

best_trial = study.best_trial

print("\n" + "=" * 80)
print("Optuna search finished")
print("=" * 80)

print("Best trial number:", best_trial.number)
print(f"Best validation {BEST_METRIC.upper()}:", best_trial.value)
print("Best raw Optuna parameters:")
print(json.dumps(best_trial.params, indent=2))

trials_dataframe = study.trials_dataframe()
trials_dataframe.to_csv(Path(OUTPUT_ROOT) / "optuna_trials.csv", index=False)

with open(Path(OUTPUT_ROOT) / "best_trial.json", "w") as handle:
    json.dump(
        {
            "trial_number": best_trial.number,
            f"validation_{BEST_METRIC}": best_trial.value,
            "params": best_trial.params,
        },
        handle,
        indent=2,
    )
```

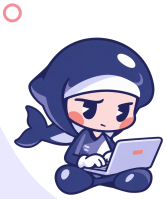




```
    train_final_model(best_trial)

    print("\nAll training finished.")
    print("Results saved to:", OUTPUT_ROOT)


if __name__ == "__main__":
    main()
```





code/historical/model_1024_classifier/evaluation_1024.py

```python
#!/usr/bin/env python3

import os
import sys
import json
from pathlib import Path

import numpy as np
import pandas as pd
import torch

from transformers import AutoConfig, AutoTokenizer
from safetensors.torch import load_file

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    confusion_matrix,
)

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT_DIR = os.path.abspath(os.path.join(SCRIPT_DIR, ".."))

LAMAR_ROOT = SCRIPT_DIR

if LAMAR_ROOT not in sys.path:
    sys.path.insert(0, LAMAR_ROOT)

from LAMAR.sequence_classification_patch import EsmForSequenceClassification


TEST_CSV = os.path.join(ROOT_DIR, "data", "1024nt_dataset", "orca_puf12_test.csv")

MODEL_WEIGHTS_PATH = os.path.join(
    SCRIPT_DIR,
    "output",
    "final_model",
    "checkpoint-240",
    "model.safetensors",
)

TOKENIZER_PATH = os.path.join(ROOT_DIR, "tokenizer")
MODEL_CONFIG_PATH = os.path.join(ROOT_DIR, "config_150M.json")

OUTPUT_DIR = os.path.join(SCRIPT_DIR, "output", "test_evaluation")


MODEL_MAX_LENGTH = 1026
POSITIONAL_EMBEDDING_TYPE = "rotary"
HIDDEN_SIZE = 768
INTERMEDIATE_SIZE = 3072
NUM_ATTENTION_HEADS = 12
```





```
NUM_HIDDEN_LAYERS = 12
NUM_LABELS = 2
HEAD_TYPE = "Linear"
FREEZE_ENCODER = False
KERNEL_SIZES = [2, 3, 5]
OCS = 32

BATCH_SIZE = 32
SEQ_COLUMN = "seq"
LABEL_COLUMN = "label"


Path(OUTPUT_DIR).mkdir(parents=True, exist_ok=True)


def check_paths():
    required_paths = {
        "TEST_CSV": TEST_CSV,
        "MODEL_WEIGHTS_PATH": MODEL_WEIGHTS_PATH,
        "TOKENIZER_PATH": TOKENIZER_PATH,
        "MODEL_CONFIG_PATH": MODEL_CONFIG_PATH,
    }
    for name, path in required_paths.items():
        if not os.path.exists(path):
            raise FileNotFoundError(f"{name} does not exist: {path}")
    print("All required paths exist.")


def load_tokenizer():
    tokenizer = AutoTokenizer.from_pretrained(
        TOKENIZER_PATH,
        model_max_length=MODEL_MAX_LENGTH,
    )

    if tokenizer.pad_token_id is None:
        if tokenizer.eos_token is not None:
            tokenizer.pad_token = tokenizer.eos_token
        else:
            raise ValueError("Tokenizer has no pad_token and no eos_token")

    print("Tokenizer vocabulary size:", len(tokenizer))
    print("Tokenizer pad_token_id:", tokenizer.pad_token_id)
    print("Tokenizer model_max_length:", tokenizer.model_max_length)

    return tokenizer


def build_model(tokenizer, device):
    config_kwargs = dict(
        vocab_size=len(tokenizer),
        pad_token_id=tokenizer.pad_token_id,
        num_labels=NUM_LABELS,
        positional_embedding_type=POSITIONAL_EMBEDDING_TYPE,
        hidden_size=HIDDEN_SIZE,
        intermediate_size=INTERMEDIATE_SIZE,
        num_attention_heads=NUM_ATTENTION_HEADS,
        num_hidden_layers=NUM_HIDDEN_LAYERS,
    )

    mask_token_id = getattr(tokenizer, "mask_token_id", None)
    if mask_token_id is not None:
```





```
        config_kwargs["mask_token_id"] = mask_token_id

    config = AutoConfig.from_pretrained(MODEL_CONFIG_PATH, **config_kwargs)

    model = EsmForSequenceClassification(
        config,
        head_type=HEAD_TYPE,
        freeze=FREEZE_ENCODER,
        kernel_sizes=KERNEL_SIZES,
        ocs=OCS,
    )

    print("Loading weights from:", MODEL_WEIGHTS_PATH)

    state_dict = load_file(MODEL_WEIGHTS_PATH, device="cpu")
    missing_keys, unexpected_keys = model.load_state_dict(state_dict, strict=False)

    print("Missing keys:", len(missing_keys))
    print("Unexpected keys:", len(unexpected_keys))

    if len(missing_keys) > 0:
        print("Missing key names (first 20):", missing_keys[:20])
    if len(unexpected_keys) > 0:
        print("Unexpected key names (first 20):", unexpected_keys[:20])

    model.to(device)
    model.eval()

    return model


def load_test_dataframe():
    df = pd.read_csv(TEST_CSV)

    if SEQ_COLUMN not in df.columns:
        raise ValueError(
            f"CSV must contain column '{SEQ_COLUMN}'. "
            f"Available columns: {list(df.columns)}"
        )

    if LABEL_COLUMN not in df.columns:
        raise ValueError(
            f"CSV must contain column '{LABEL_COLUMN}'. "
            f"Available columns: {list(df.columns)}"
        )

    df = df[[SEQ_COLUMN, LABEL_COLUMN]].copy()
    df = df.dropna(subset=[SEQ_COLUMN, LABEL_COLUMN])
    df[SEQ_COLUMN] = df[SEQ_COLUMN].astype(str)
    df[LABEL_COLUMN] = df[LABEL_COLUMN].astype(int)

    print("Test dataframe shape:", df.shape)
    print("Test label distribution:", df[LABEL_COLUMN].value_counts().to_dict())

    return df


@torch.no_grad()
def run_inference(model, tokenizer, sequences, device):
    all_logits = []

```





```
    total = len(sequences)

    for start in range(0, total, BATCH_SIZE):
        end = min(start + BATCH_SIZE, total)
        batch_sequences = sequences[start:end]

        encoded = tokenizer(
            batch_sequences,
            truncation=True,
            max_length=MODEL_MAX_LENGTH,
            padding=True,
            return_tensors="pt",
        )

        input_ids = encoded["input_ids"].to(device)
        attention_mask = encoded["attention_mask"].to(device)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask)

        logits = outputs.logits if hasattr(outputs, "logits") else outputs[0]

        all_logits.append(logits.detach().cpu())

        print(f"Inference progress: {end}/{total}")

    return torch.cat(all_logits, dim=0)


def numpy_softmax(logits):
    logits = np.asarray(logits, dtype=np.float64)
    logits = logits - np.max(logits, axis=1, keepdims=True)
    exp_logits = np.exp(logits)
    return exp_logits / np.sum(exp_logits, axis=1, keepdims=True)


def compute_metrics_from_logits(logits, labels):
    if isinstance(logits, torch.Tensor):
        logits = logits.detach().cpu().numpy()

    logits = np.asarray(logits, dtype=np.float64)
    labels = np.asarray(labels).reshape(-1)

    probabilities = numpy_softmax(logits)
    positive_probabilities = probabilities[:, 1]
    predicted_classes = np.argmax(probabilities, axis=1)

    metrics = {
        "accuracy": float(accuracy_score(labels, predicted_classes)),
        "precision": float(precision_score(labels, predicted_classes, zero_division=0)),
        "recall": float(recall_score(labels, predicted_classes, zero_division=0)),
        "f1": float(f1_score(labels, predicted_classes, zero_division=0)),
    }

    try:
        metrics["roc_auc"] = float(roc_auc_score(labels, positive_probabilities))
    except Exception:
        metrics["roc_auc"] = float("nan")

    try:
        metrics["pr_auc"] = float(average_precision_score(labels, positive_probabilities))
    except Exception:
```





```
        metrics["pr_auc"] = float("nan")

    tn, fp, fn, tp = confusion_matrix(labels, predicted_classes, labels=[0, 1]).ravel()

    metrics["true_negative"] = int(tn)
    metrics["false_positive"] = int(fp)
    metrics["false_negative"] = int(fn)
    metrics["true_positive"] = int(tp)

    metrics["num_samples"] = int(len(labels))
    metrics["num_positive"] = int((labels == 1).sum())
    metrics["num_negative"] = int((labels == 0).sum())

    return metrics, probabilities, predicted_classes


def main():
    check_paths()

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("Using device:", device)

    tokenizer = load_tokenizer()
    model = build_model(tokenizer, device)

    df = load_test_dataframe()

    sequences = df[SEQ_COLUMN].tolist()
    labels = df[LABEL_COLUMN].to_numpy(dtype=np.int64)

    logits = run_inference(model, tokenizer, sequences, device)

    metrics, probabilities, predicted_classes = compute_metrics_from_logits(
        logits,
        labels,
    )

    print("\n" + "=" * 80)
    print("Test metrics")
    print("=" * 80)
    print(json.dumps(metrics, indent=2))

    logits_np = logits.detach().cpu().numpy()

    predictions_df = pd.DataFrame(
        {
            "seq": sequences,
            "label": labels,
            "predicted_class": predicted_classes,
            "prob_class_0": probabilities[:, 0],
            "prob_class_1": probabilities[:, 1],
            "logit_class_0": logits_np[:, 0],
            "logit_class_1": logits_np[:, 1],
        }
    )

    predictions_path = Path(OUTPUT_DIR) / "test_predictions.csv"
    metrics_path = Path(OUTPUT_DIR) / "test_metrics.json"

    predictions_df.to_csv(predictions_path, index=False)

```





```
    with open(metrics_path, "w") as handle:
        json.dump(metrics, handle, indent=2)

    print("\nSaved predictions to:", predictions_path)
    print("Saved metrics to:", metrics_path)

    print("\nConfusion matrix:")
    print(f"  TN={metrics['true_negative']}  FP={metrics['false_positive']}")
    print(f"  FN={metrics['false_negative']}  TP={metrics['true_positive']}")


if __name__ == "__main__":
    main()
```





code/historical/model_101_classifier/train_orca_ranking_101.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import os

os.environ.setdefault("CUDA_VISIBLE_DEVICES", "0,1,2,3")
os.environ.setdefault("TOKENIZERS_PARALLELISM", "false")

import sys
import json
import random
import gc
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from safetensors.torch import load_file, save_file

from transformers import (
    AutoConfig,
    AutoTokenizer,
    set_seed as hf_set_seed,
    get_cosine_schedule_with_warmup,
)

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT_DIR = os.path.abspath(os.path.join(SCRIPT_DIR, ".."))

LAMAR_ROOT = SCRIPT_DIR

if LAMAR_ROOT not in sys.path:
    sys.path.insert(0, LAMAR_ROOT)

from LAMAR.sequence_classification_patch import EsmForSequenceClassification


TRAIN_JSON = os.path.join(ROOT_DIR, "data", "101nt_dataset", "orca_puf12_train.json")
VALID_JSON = os.path.join(ROOT_DIR, "data", "101nt_dataset", "orca_puf12_valid.json")

INIT_MODEL = os.path.join(
    ROOT_DIR,
    "model_continual_pretrain",
    "output",
    "transcriptome_pretrain",
    "checkpoint-100000",
    "model.safetensors",
)

TOKENIZER_PATH = os.path.join(ROOT_DIR, "tokenizer")
MODEL_CONFIG_PATH = os.path.join(ROOT_DIR, "config_150M.json")
OUTPUT_DIR = os.path.join(SCRIPT_DIR, "output")

Path(OUTPUT_DIR).mkdir(parents=True, exist_ok=True)

```





```

SEED = 42
WINDOW_LENGTH = 101
HALF_WINDOW = WINDOW_LENGTH // 2
MODEL_MAX_LENGTH = 103
CENTER_TOKEN_INDEX = 51

HIDDEN_SIZE = 768
INTERMEDIATE_SIZE = 3072
NUM_ATTENTION_HEADS = 12
NUM_HIDDEN_LAYERS = 12
NUM_LABELS = 1
POSITIONAL_EMBEDDING_TYPE = "rotary"
FREEZE_ENCODER = False
NUM_EPOCHS = 8
LEARNING_RATE = 5e-6
WEIGHT_DECAY = 0.01
GRADIENT_ACCUMULATION = 8
BATCH_SIZE_PER_GPU = 1
NUM_WORKERS = 4
WARMUP_RATIO = 0.05
SAVE_BEST_METRIC = "hit10"


def set_all_seeds(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    hf_set_seed(seed)


set_all_seeds(SEED)


def check_paths():
    paths = {
        "TRAIN_JSON": TRAIN_JSON,
        "VALID_JSON": VALID_JSON,
        "INIT_MODEL": INIT_MODEL,
        "TOKENIZER_PATH": TOKENIZER_PATH,
        "MODEL_CONFIG_PATH": MODEL_CONFIG_PATH,
    }
    for name, path in paths.items():
        if not os.path.exists(path):
            raise FileNotFoundError(f"{name} missing: {path}")
    print("All paths exist.")


check_paths()
print("CUDA available:", torch.cuda.is_available())
print("GPU count:", torch.cuda.device_count())


print("Loading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(
    TOKENIZER_PATH,
    model_max_length=MODEL_MAX_LENGTH,
)
print("Vocabulary size:", len(tokenizer))
```





```

def load_json(path):
    with open(path, "r") as f:
        data = json.load(f)
    print(path, "samples:", len(data))
    return data


train_records = load_json(TRAIN_JSON)
valid_records = load_json(VALID_JSON)


def normalize_sequence(seq):
    seq = seq.upper()
    seq = seq.replace("U", "T")
    return seq


def get_center_window(sequence, center_position):
    start = center_position - HALF_WINDOW
    end = center_position + HALF_WINDOW + 1

    left_pad = 0
    right_pad = 0

    if start < 0:
        left_pad = -start
        start = 0

    if end > len(sequence):
        right_pad = end - len(sequence)
        end = len(sequence)

    window = "N" * left_pad + sequence[start:end] + "N" * right_pad
    assert len(window) == WINDOW_LENGTH
    return window


def extract_candidates(record):
    seq = normalize_sequence(record["seq"])
    labels = record["label"]

    assert len(seq) == len(labels), (len(seq), len(labels))

    candidates = []
    for pos, base in enumerate(seq):
        if base != "C":
            continue
        candidates.append(
            {
                "position": pos,
                "window": get_center_window(seq, pos),
                "raw_label": float(labels[pos]),
                "label": 1.0 if labels[pos] > 0 else 0.0,
            }
        )
    return candidates


class OrcaRankingDataset(Dataset):
```





```
    def __init__(self, records):
        self.records = records

    def __len__(self):
        return len(self.records)

    def __getitem__(self, idx):
        record = self.records[idx]
        candidates = extract_candidates(record)
        if len(candidates) == 0:
            return None

        windows = [x["window"] for x in candidates]
        labels = torch.tensor([x["label"] for x in candidates], dtype=torch.float)
        positions = [x["position"] for x in candidates]

        return {
            "windows": windows,
            "labels": labels,
            "positions": positions,
        }


def ranking_collate_fn(batch):
    batch = [item for item in batch if item is not None]
    if len(batch) == 0:
        return None
    return batch


def tokenize_windows(windows, device):
    encoded = tokenizer(
        windows,
        padding=True,
        truncation=True,
        max_length=MODEL_MAX_LENGTH,
        return_tensors="pt",
    )
    encoded = {k: v.to(device) for k, v in encoded.items()}
    return encoded


class CenterTokenScoreHead(nn.Module):
    def __init__(self, config, center_index):
        super().__init__()
        self.center_index = center_index
        self.dropout = nn.Dropout(config.hidden_dropout_prob)
        self.dense = nn.Linear(config.hidden_size, config.hidden_size)
        self.out_proj = nn.Linear(config.hidden_size, 1)

    def forward(self, hidden_states):
        x = hidden_states[:, self.center_index, :]
        x = self.dropout(x)
        x = self.dense(x)
        x = torch.tanh(x)
        x = self.dropout(x)
        score = self.out_proj(x)
        return score.squeeze(-1)


def build_model():
```





```
    config = AutoConfig.from_pretrained(
        MODEL_CONFIG_PATH,
        vocab_size=len(tokenizer),
        pad_token_id=tokenizer.pad_token_id,
        num_labels=1,
        problem_type="regression",
        hidden_size=HIDDEN_SIZE,
        intermediate_size=INTERMEDIATE_SIZE,
        num_attention_heads=NUM_ATTENTION_HEADS,
        num_hidden_layers=NUM_HIDDEN_LAYERS,
        positional_embedding_type=POSITIONAL_EMBEDDING_TYPE,
    )

    print("Building LAMAR model...")
    model = EsmForSequenceClassification(
        config,
        head_type="Linear",
        freeze=FREEZE_ENCODER,
        kernel_sizes=[2, 3, 5],
        ocs=32,
    )

    model.classifier = CenterTokenScoreHead(config, CENTER_TOKEN_INDEX)

    print("Loading previous ORCA binary model:")
    print(INIT_MODEL)

    state_dict = load_file(INIT_MODEL, device="cpu")
    state_dict = {k.replace("module.", "", 1): v for k, v in state_dict.items()}
    state_dict = {k: v for k, v in state_dict.items() if not k.startswith("classifier.")}

    missing, unexpected = model.load_state_dict(state_dict, strict=False)

    encoder_missing = [k for k in missing if not k.startswith("classifier.")]

    print("Missing keys:", len(missing))
    print("Unexpected keys:", len(unexpected))

    if len(encoder_missing) > 0:
        print("Warning: missing encoder keys:")
        print(encoder_missing[:20])
    else:
        print("Encoder weights loaded successfully.")

    return model, config


class MultiPositiveRankingLoss(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, scores, labels):
        positive_mask = labels > 0
        if positive_mask.sum() == 0:
            return None
        positive_scores = scores[positive_mask]
        all_logsumexp = torch.logsumexp(scores, dim=0)
        positive_logsumexp = torch.logsumexp(positive_scores, dim=0)
        loss = -(positive_logsumexp - all_logsumexp)
        return loss

```





```
301
302 def calculate_rank_metrics(scores, labels):
303     order = np.argsort(-scores)
304     sorted_labels = labels[order]
305
306     positive_indices = np.where(sorted_labels > 0)[0]
307     if len(positive_indices) == 0:
308         return None
309
310     best_rank = positive_indices[0] + 1
311     hit = {}
312     for k in [1, 5, 10, 20]:
313         hit[k] = int(np.any(sorted_labels[:k] > 0))
314
315     recall = {}
316     total_positive = labels.sum()
317     for k in [5, 10, 20]:
318         recall[k] = sorted_labels[:k].sum() / max(total_positive, 1)
319
320     mrr = 1.0 / best_rank
321     return {
322         "rank": best_rank,
323         "hit": hit,
324         "recall": recall,
325         "mrr": mrr,
326     }
327
328
329 def forward_transcript(model, transcript, device):
330     windows = transcript["windows"]
331     labels = transcript["labels"].to(device)
332
333     encoded = tokenize_windows(windows, device)
334
335     outputs = model(**encoded, output_hidden_states=True)
336
337     if isinstance(model, torch.nn.DataParallel):
338         classifier = model.module.classifier
339     else:
340         classifier = model.classifier
341
342     hidden = outputs.hidden_states[-1]
343     scores = classifier(hidden)
344
345     return scores, labels
346
347
348 def train_one_epoch(model, loader, optimizer, scheduler, device, epoch):
349     model.train()
350     loss_function = MultiPositiveRankingLoss()
351
352     total_loss = 0.0
353     valid_steps = 0
354     optimizer.zero_grad()
355
356     for step, batch in enumerate(loader):
357         if batch is None:
358             continue
359
360         batch_loss = 0.0
361         transcript_count = 0
```





```

        for transcript in batch:
            scores, labels = forward_transcript(model, transcript, device)
            loss = loss_function(scores, labels)
            if loss is None:
                continue
            batch_loss += loss
            transcript_count += 1

        if transcript_count == 0:
            continue

        batch_loss = batch_loss / transcript_count
        log_loss = batch_loss.item()
        (batch_loss / GRADIENT_ACCUMULATION).backward()

        if (step + 1) % GRADIENT_ACCUMULATION == 0:
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad()

        total_loss += log_loss
        valid_steps += 1

        if step % 20 == 0:
            print(f"Epoch {epoch} Step {step} Loss {log_loss:.5f}")

    return total_loss / max(valid_steps, 1)


@torch.no_grad()
def calculate_transcript_metrics(model, transcript, device):
    scores, labels = forward_transcript(model, transcript, device)
    scores = scores.detach().cpu().numpy()
    labels = labels.detach().cpu().numpy()
    return calculate_rank_metrics(scores, labels)


@torch.no_grad()
def evaluate(model, loader, device):
    model.eval()
    all_metrics = []

    for batch in loader:
        if batch is None:
            continue
        for transcript in batch:
            result = calculate_transcript_metrics(model, transcript, device)
            if result is not None:
                all_metrics.append(result)

    if len(all_metrics) == 0:
        return {}

    metrics = {}
    for k in [1, 5, 10, 20]:
        metrics[f"hit{k}"] = float(np.mean([x["hit"][k] for x in all_metrics]))
    for k in [5, 10, 20]:
        metrics[f"recall{k}"] = float(np.mean([x["recall"][k] for x in all_metrics]))
    metrics["mrr"] = float(np.mean([x["mrr"] for x in all_metrics]))
```





```

    ranks = [x["rank"] for x in all_metrics]
    metrics["mean_best_rank"] = float(np.mean(ranks))
    metrics["median_best_rank"] = float(np.median(ranks))
    metrics["num_transcripts"] = len(all_metrics)

    return metrics


def build_optimizer(model):
    no_decay = ["bias", "LayerNorm.weight"]
    optimizer_grouped_parameters = [
        {
            "params": [
                p
                for n, p in model.named_parameters()
                if not any(nd in n for nd in no_decay)
            ],
            "weight_decay": WEIGHT_DECAY,
        },
        {
            "params": [
                p for n, p in model.named_parameters() if any(nd in n for nd in no_decay)
            ],
            "weight_decay": 0.0,
        },
    ]

    optimizer = torch.optim.AdamW(optimizer_grouped_parameters, lr=LEARNING_RATE)
    return optimizer


def save_checkpoint(
    model,
    config,
    tokenizer,
    optimizer,
    scheduler,
    output_dir,
    epoch,
    best_metric,
    global_step,
):
    Path(output_dir).mkdir(parents=True, exist_ok=True)

    model_to_save = model
    if isinstance(model, torch.nn.DataParallel):
        model_to_save = model.module

    state_dict = {k: v.cpu() for k, v in model_to_save.state_dict().items()}
    save_file(state_dict, os.path.join(output_dir, "model.safetensors"))

    config.save_pretrained(output_dir)
    tokenizer.save_pretrained(output_dir)

    torch.save(
        optimizer.state_dict(),
        os.path.join(output_dir, "optimizer.pt"),
    )
    torch.save(
        scheduler.state_dict(),
```





```
        os.path.join(output_dir, "scheduler.pt"),
    )

    training_state = {
        "epoch": epoch,
        "global_step": global_step,
        "best_metric": best_metric,
        "save_best_metric": SAVE_BEST_METRIC,
        "learning_rate": LEARNING_RATE,
        "weight_decay": WEIGHT_DECAY,
        "gradient_accumulation": GRADIENT_ACCUMULATION,
        "num_epochs": NUM_EPOCHS,
        "seed": SEED,
        "window_length": WINDOW_LENGTH,
        "model_max_length": MODEL_MAX_LENGTH,
        "center_token_index": CENTER_TOKEN_INDEX,
        "init_model": INIT_MODEL,
        "train_json": TRAIN_JSON,
        "valid_json": VALID_JSON,
    }
    with open(os.path.join(output_dir, "training_state.json"), "w") as f:
        json.dump(training_state, f, indent=2)

    print("Saved checkpoint:", output_dir)


def main():
    print("\nCreating datasets...")

    train_dataset = OrcaRankingDataset(train_records)
    valid_dataset = OrcaRankingDataset(valid_records)

    train_loader = DataLoader(
        train_dataset,
        batch_size=BATCH_SIZE_PER_GPU,
        shuffle=True,
        num_workers=NUM_WORKERS,
        collate_fn=ranking_collate_fn,
        pin_memory=True,
    )

    valid_loader = DataLoader(
        valid_dataset,
        batch_size=1,
        shuffle=False,
        num_workers=NUM_WORKERS,
        collate_fn=ranking_collate_fn,
        pin_memory=True,
    )

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    print("\nBuilding model...")
    model, config = build_model()
    model.to(device)

    if torch.cuda.device_count() > 1:
        print("Using DataParallel:", torch.cuda.device_count(), "GPUs")
        model = torch.nn.DataParallel(model)

    optimizer = build_optimizer(model)
```





```
    total_steps = len(train_loader) * NUM_EPOCHS // GRADIENT_ACCUMULATION
    warmup_steps = int(total_steps * WARMUP_RATIO)
    scheduler = get_cosine_schedule_with_warmup(
        optimizer,
        num_warmup_steps=warmup_steps,
        num_training_steps=total_steps,
    )

    print("Total optimizer steps:", total_steps)
    print("Warmup steps:", warmup_steps)

    best_metric = -1
    global_step = 0

    print("\nStart training...")

    for epoch in range(1, NUM_EPOCHS + 1):
        print("\n====================")
        print("Epoch", epoch)

        train_loss = train_one_epoch(
            model, train_loader, optimizer, scheduler, device, epoch
        )
        global_step = epoch * (len(train_loader) // GRADIENT_ACCUMULATION)
        print("Training loss:", train_loss)

        valid_metrics = evaluate(model, valid_loader, device)
        print("\nValidation metrics:")
        for k, v in valid_metrics.items():
            print(f"{k}: {v}")

        current_metric = valid_metrics.get(SAVE_BEST_METRIC, 0)
        if current_metric > best_metric:
            best_metric = current_metric
            print("New best model:", best_metric)
            save_checkpoint(
                model,
                config,
                tokenizer,
                optimizer,
                scheduler,
                os.path.join(OUTPUT_DIR, "best_model"),
                epoch,
                best_metric,
                global_step,
            )

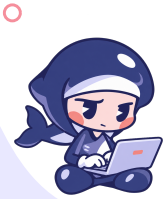
        save_checkpoint(
            model,
            config,
            tokenizer,
            optimizer,
            scheduler,
            os.path.join(OUTPUT_DIR, "last_model"),
            epoch,
            best_metric,
            global_step,
        )

        gc.collect()
```





```
606         if torch.cuda.is_available():
607             torch.cuda.empty_cache()
608 
609     print("\nTraining finished.")
610     print("Best", SAVE_BEST_METRIC, ":", best_metric)
611 
612 
613 if __name__ == "__main__":
614     main()
```





code/historical/model_101_classifier/evaluation_top_ranking.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

"""
Evaluation script for LAMAR ORCA ranking model.

Outputs:
    hit@1
    hit@5
    hit@10
    hit@20
    recall@5
    recall@10
    recall@20
    MRR
    mean best rank
    median best rank
"""

import os
import sys
import json
import random
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn

from torch.utils.data import Dataset, DataLoader

from safetensors.torch import load_file

from transformers import (
    AutoConfig,
    AutoTokenizer,
)

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT_DIR = os.path.abspath(os.path.join(SCRIPT_DIR, ".."))

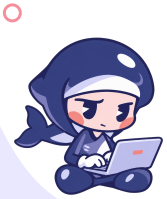
LAMAR_ROOT = SCRIPT_DIR

if LAMAR_ROOT not in sys.path:
    sys.path.insert(0, LAMAR_ROOT)

from LAMAR.sequence_classification_patch import (
    EsmForSequenceClassification,
)


# ============================================================
# PATHS
# ============================================================

TEST_JSON = os.path.join(ROOT_DIR, "data", "101nt_dataset", "orca_puf12_test.json")
```





```
57
58  MODEL_PATH = os.path.join(SCRIPT_DIR, "output", "best_model", "model.safetensors")
59
60  TOKENIZER_PATH = os.path.join(ROOT_DIR, "tokenizer")
61
62  MODEL_CONFIG_PATH = os.path.join(ROOT_DIR, "config_150M.json")
63
64
65  # ============================================================
66  # PARAMETERS
67  # ============================================================
68
69  WINDOW_LENGTH = 101
70  HALF_WINDOW = 50
71
72  MODEL_MAX_LENGTH = 103
73
74  CENTER_TOKEN_INDEX = 51
75
76
77  HIDDEN_SIZE = 768
78  INTERMEDIATE_SIZE = 3072
79  NUM_ATTENTION_HEADS = 12
80  NUM_HIDDEN_LAYERS = 12
81
82  POSITIONAL_EMBEDDING_TYPE = "rotary"
83
84
85  NUM_WORKERS = 4
86
87
88  # ============================================================
89  # SEED
90  # ============================================================
91
92
93  def seed_everything(seed=42):
94
95      random.seed(seed)
96      np.random.seed(seed)
97
98      torch.manual_seed(seed)
99
100     if torch.cuda.is_available():
101         torch.cuda.manual_seed_all(seed)
102
103
104 seed_everything()
105
106
107 # ============================================================
108 # LOAD DATA
109 # ============================================================
110
111
112 def load_json(path):
113
114     with open(path, "r") as f:
115         data = json.load(f)
116
117     print("Loaded:", path, "samples:", len(data))
```





```
    return data


records = load_json(TEST_JSON)


# ============================================================
# TOKENIZER
# ============================================================


print("Loading tokenizer")


tokenizer = AutoTokenizer.from_pretrained(
    TOKENIZER_PATH, model_max_length=MODEL_MAX_LENGTH
)


print("vocab:", len(tokenizer))


# ============================================================
# SEQUENCE PROCESS
# ============================================================


def normalize_sequence(seq):

    seq = seq.upper()
    seq = seq.replace("U", "T")

    return seq


def get_center_window(sequence, center_position):

    start = center_position - HALF_WINDOW

    end = center_position + HALF_WINDOW + 1

    left_pad = 0
    right_pad = 0

    if start < 0:

        left_pad = -start
        start = 0

    if end > len(sequence):

        right_pad = end - len(sequence)
        end = len(sequence)

    window = "N" * left_pad + sequence[start:end] + "N" * right_pad

    assert len(window) == WINDOW_LENGTH

    return window

```





```
# ============================================================
# CANDIDATE EXTRACTION
# ============================================================


def extract_candidates(record):

    seq = normalize_sequence(record["seq"])

    labels = record["label"]

    candidates = []

    for pos, base in enumerate(seq):

        if base != "C":
            continue

        candidates.append(
            {
                "position": pos,
                "window": get_center_window(seq, pos),
                "label": float(labels[pos] > 0),
                "raw_label": float(labels[pos]),
            }
        )

    return candidates


# ============================================================
# DATASET
# ============================================================


class RankingDataset(Dataset):

    def __init__(self, records):

        self.records = records

    def __len__(self):

        return len(self.records)

    def __getitem__(self, index):

        candidates = extract_candidates(self.records[index])

        if len(candidates) == 0:

            return None

        return {
            "windows": [x["window"] for x in candidates],
            "labels": torch.tensor([x["label"] for x in candidates], dtype=torch.float),
            "positions": [x["position"] for x in candidates],
            "raw_labels": [x["raw_label"] for x in candidates],
        }

```





```
240
241 def collate_fn(batch):
242
243     batch = [x for x in batch if x is not None]
244
245     if len(batch) == 0:
246
247         return None
248
249     return batch
250
251
252 # ====================================================
253 # MODEL HEAD
254 # ====================================================
255
256
257 class CenterTokenScoreHead(nn.Module):
258
259     def __init__(self, config, center_index):
260
261         super().__init__()
262
263         self.center_index = center_index
264
265         self.dropout = nn.Dropout(config.hidden_dropout_prob)
266
267         self.dense = nn.Linear(config.hidden_size, config.hidden_size)
268
269         self.out_proj = nn.Linear(config.hidden_size, 1)
270
271     def forward(self, hidden_states):
272
273         x = hidden_states[:, self.center_index, :]
274
275         x = self.dropout(x)
276
277         x = self.dense(x)
278
279         x = torch.tanh(x)
280
281         x = self.dropout(x)
282
283         score = self.out_proj(x)
284
285         return score.squeeze(-1)
286
287
288 # ====================================================
289 # BUILD MODEL
290 # ====================================================
291
292
293 def build_model():
294
295     config = AutoConfig.from_pretrained(
296         MODEL_CONFIG_PATH,
297         vocab_size=len(tokenizer),
298         pad_token_id=tokenizer.pad_token_id,
299         num_labels=1,
300         problem_type="regression",
```





```
        hidden_size=HIDDEN_SIZE,
        intermediate_size=INTERMEDIATE_SIZE,
        num_attention_heads=NUM_ATTENTION_HEADS,
        num_hidden_layers=NUM_HIDDEN_LAYERS,
        positional_embedding_type=POSITIONAL_EMBEDDING_TYPE,
    )

    model = EsmForSequenceClassification(
        config, head_type="Linear", freeze=False, kernel_sizes=[2, 3, 5], ocs=32
    )

    model.classifier = CenterTokenScoreHead(config, CENTER_TOKEN_INDEX)

    print("Loading model:", MODEL_PATH)

    state_dict = load_file(MODEL_PATH, device="cpu")

    model.load_state_dict(state_dict, strict=True)

    print("Model loaded")

    return model


# ============================================================
# TOKENIZE
# ============================================================


def tokenize_windows(windows, device):

    encoded = tokenizer(
        windows,
        padding=True,
        truncation=True,
        max_length=MODEL_MAX_LENGTH,
        return_tensors="pt",
    )

    return {k: v.to(device) for k, v in encoded.items()}


# ============================================================
# FORWARD
# ============================================================


@torch.no_grad()
def predict_scores(model, transcript, device):

    encoded = tokenize_windows(transcript["windows"], device)

    outputs = model(**encoded, output_hidden_states=True)

    hidden = outputs.hidden_states[-1]

    scores = model.classifier(hidden)

    return (scores.cpu().numpy(), transcript["labels"].numpy())


```





```
# ============================================================
# METRICS
# ============================================================

def calculate_metrics(scores, labels):

    order = np.argsort(-scores)

    sorted_labels = labels[order]

    positive = np.where(sorted_labels > 0)[0]

    if len(positive) == 0:

        return None

    best_rank = positive[0] + 1

    result = {}

    for k in [1, 5, 10, 20]:

        result[f"hit{k}"] = int(np.any(sorted_labels[:k] > 0))

    total_positive = labels.sum()

    for k in [5, 10, 20]:

        result[f"recall{k}"] = sorted_labels[:k].sum() / max(total_positive, 1)

    result["mrr"] = 1.0 / best_rank

    result["rank"] = best_rank

    return result


# ============================================================
# EVALUATION
# ============================================================


@torch.no_grad()
def evaluate(model, loader, device):

    model.eval()

    all_results = []

    for batch in loader:

        if batch is None:
            continue

        for transcript in batch:

            scores, labels = predict_scores(model, transcript, device)

            metric = calculate_metrics(scores, labels)

```





```
            if metric:

                all_results.append(metric)

    final = {}

    for k in [1, 5, 10, 20]:

        final[f"hit{k}"] = np.mean([x[f"hit{k}"] for x in all_results])

    for k in [5, 10, 20]:

        final[f"recall{k}"] = np.mean([x[f"recall{k}"] for x in all_results])

    final["MRR"] = np.mean([x["mrr"] for x in all_results])

    ranks = [x["rank"] for x in all_results]

    final["mean_best_rank"] = np.mean(ranks)

    final["median_best_rank"] = np.median(ranks)

    final["num_transcripts"] = len(all_results)

    return final


# ============================================================
# MAIN
# ============================================================


def main():

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    print("Device:", device)

    dataset = RankingDataset(records)

    loader = DataLoader(
        dataset,
        batch_size=1,
        shuffle=False,
        num_workers=NUM_WORKERS,
        collate_fn=collate_fn,
    )

    model = build_model()

    model.to(device)

    metrics = evaluate(model, loader, device)

    print("\n======================")
    print("FINAL RESULTS")
    print("======================")
```





```
    for k, v in metrics.items():

        print(f"{k}: {v:.6f}" if isinstance(v, float) else f"{k}: {v}")


if __name__ == "__main__":

    main()
```





code/historical/config_150M.json

```
{
  "_name_or_path": "esm2_t30_150M_UR50D",
  "architectures": [
    "EsmForMaskedLM"
  ],
  "attention_probs_dropout_prob": 0.0,
  "classifier_dropout": null,
  "emb_layer_norm_before": false,
  "esmfold_config": null,
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.0,
  "hidden_size": 640,
  "initializer_range": 0.02,
  "intermediate_size": 2560,
  "is_folding_model": false,
  "layer_norm_eps": 1e-05,
  "mask_token_id": 32,
  "max_position_embeddings": 1026,
  "model_type": "esm",
  "num_attention_heads": 20,
  "num_hidden_layers": 30,
  "pad_token_id": 1,
  "position_embedding_type": "rotary",
  "token_dropout": true,
  "torch_dtype": "float32",
  "transformers_version": "4.25.0.dev0",
  "use_cache": true,
  "vocab_list": null,
  "vocab_size": 33
}
```





code/historical/tokenizer/tokenizer_config.json

```
{
  "clean_up_tokenization_spaces": true,
  "model_max_length": 201,
  "tokenizer_class": "EsmTokenizer"
}
```





code/historical/tokenizer/special_tokens_map.json

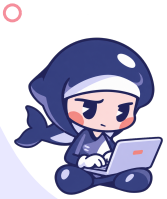
```
{
  "cls_token": "<cls>",
  "eos_token": "<eos>",
  "mask_token": "<mask>",
  "pad_token": "<pad>",
  "unk_token": "<unk>"
}
```





code/historical/tokenizer/vocab.txt

```
<cls>
<pad>
<eos>
<unk>
<mask>
A
T
C
G
N
```





scripts/validate_repository.py

```python
#!/usr/bin/env python3
"""Audit the published bundle using only Python's standard library.

This checks saved data and predictions; it does not run or validate a model.
Outputs are deterministic and contain repository-relative paths only.
"""

import argparse
import ast
import collections
import csv
import hashlib
import itertools
import json
import math
from pathlib import Path
import sys


SPLITS = ("train", "valid", "test")
EXPECTED = {
    "train": (3835, 2643, 1192),
    "valid": (213, 150, 63),
    "test": (212, 140, 72),
}
RESULT_BASE = Path("results/model_1024_classifier/1024_orca_puf12")


def sha256(path):
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def read_csv(path, delimiter=","):
    with path.open(newline="", encoding="utf-8") as handle:
        return list(csv.DictReader(handle, delimiter=delimiter))


def read_json(path):
    with path.open(encoding="utf-8") as handle:
        return json.load(handle)


def average_precision(labels, scores):
    """Non-interpolated AP, grouping tied scores at the same threshold."""
    positive_count = sum(labels)
    if not positive_count:
        return 0.0
    ordered = sorted(zip(scores, labels), reverse=True)
    true_positives = seen = 0
    result = 0.0
    for _, group in itertools.groupby(ordered, key=lambda pair: pair[0]):
        members = list(group)
```





```
        new_positives = sum(label for _, label in members)
        true_positives += new_positives
        seen += len(members)
        result += (new_positives / positive_count) * (true_positives / seen)
    return result


def roc_auc(labels, scores):
    """Pairwise ROC AUC with half credit for tied positive-negative pairs."""
    positive_count = sum(labels)
    negative_count = len(labels) - positive_count
    if not positive_count or not negative_count:
        return None
    negative_below = 0
    credit = 0.0
    for _, group in itertools.groupby(sorted(zip(scores, labels)), key=lambda x: x[0]):
        members = list(group)
        positives = sum(label for _, label in members)
        negatives = len(members) - positives
        credit += positives * (negative_below + 0.5 * negatives)
        negative_below += negatives
    return credit / (positive_count * negative_count)


def candidate_contexts(records):
    contexts = collections.Counter()
    for record in records:
        sequence = record["seq"].upper().replace("U", "T")
        padded = "N" * 50 + sequence + "N" * 50
        for position, base in enumerate(sequence):
            if base == "C":
                contexts[padded[position : position + 101]] += 1
    return contexts


def audit(root):
    report = {
        "scope": "Saved-artifact integrity and descriptive audit; no model execution",
        "errors": [],
        "warnings": [],
    }

    def check(condition, message):
        if not condition:
            report["errors"].append(message)

    manifest = read_json(root / "provenance/import-manifest.json")
    entries = manifest["entries"]
    published_paths = []
    byte_identical = transformed = 0
    for entry in entries:
        relative = Path(entry["published_path"])
        check(
            not relative.is_absolute() and ".." not in relative.parts,
            "Unsafe manifest path",
        )
        if relative.is_absolute() or ".." in relative.parts:
            continue
        published_paths.append(relative.as_posix())
        path = root / relative
        if entry.get("storage") == "git_history":
```





```
            check(
                relative.as_posix().startswith("provenance/source-documents/"),
                "Only removed prose drafts may be recorded as historical",
            )
            check(not path.exists(), f"Obsolete draft was restored: {relative}")
            check(
                entry.get("repository_revision")
                == "17a00c641f0a7ce726d65e16c2cca4b58a841431",
                "Unexpected historical prose revision",
            )
            continue
        check(path.is_file(), f"Missing manifest file: {relative}")
        if not path.is_file():
            continue
        digest = sha256(path)
        check(
            digest == entry["published_sha256"],
            f"Published checksum mismatch: {relative}",
        )
        if entry["transformation"] == "byte-identical copy":
            byte_identical += 1
            check(
                digest == entry["source_sha256"], f"Source checksum mismatch: {relative}"
            )
        else:
            transformed += 1
    check(
        len(published_paths) == len(set(published_paths)),
        "Duplicate published path in import manifest",
    )
    report["import_manifest"] = {
        "entries": len(entries),
        "historical_prose_entries": sum(
            e.get("storage") == "git_history" for e in entries
        ),
        "byte_identical_entries": byte_identical,
        "transformed_entries": transformed,
        "source_archive_available_required": False,
        "note": "Checks published bytes against recorded digests; transformed originals require th
e source archive for independent verification.",
    }

    inventory = read_json(root / "provenance/archive-inventory.json")
    inventory_entries = inventory["entries"]
    inventory_published = {
        entry["published_path"]
        for entry in inventory_entries
        if entry.get("published_path")
    }
    check(
        inventory_published == set(published_paths),
        "Archive inventory and publication manifest disagree",
    )
    members = [entry["source_member"] for entry in inventory_entries]
    check(
        len(members) == len(set(members)), "Duplicate source member in archive inventory"
    )
    report["archive_inventory"] = {
        "members": len(inventory_entries),
        "published_members": len(inventory_published),
        "unpublished_members_retained_in_original_archive": sum(
```





```
            not entry.get("published_path") for entry in inventory_entries
        ),
        "model_weight_members_retained_in_original_archive": sum(
            name.endswith(".safetensors") for name in members
        ),
        "note": "Inventory describes archive contents; large unpublished members are not read or c
.
hecked by this lightweight validator.",
    }

    json_count = 0
    for path in sorted(root.rglob("*.json")):
        if ".git" in path.parts:
            continue
        try:
            read_json(path)
            json_count += 1
        except (ValueError, OSError) as error:
            check(False, f"Invalid JSON: {path.relative_to(root)}: {error}")
    python_count = 0
    for directory in (root / "code", root / "scripts"):
        for path in sorted(directory.rglob("*.py")):
            try:
                ast.parse(path.read_text(encoding="utf-8"), filename=path.name)
                python_count += 1
            except (SyntaxError, UnicodeError) as error:
                check(False, f"Invalid Python syntax: {path.relative_to(root)}: {error}")
    report["syntax"] = {
        "json_files_parsed": json_count,
        "python_files_parsed_without_execution": python_count,
    }

    regions = {}
    contexts = {}
    sequence_label_occurrences = collections.defaultdict(list)
    report["datasets"] = {}
    for split in SPLITS:
        rows = read_csv(root / f"data/1024nt_dataset/orca_puf12_{split}.csv")
        records = read_json(root / f"data/101nt_dataset/orca_puf12_{split}.json")
        check(
            bool(rows) and set(rows[0]) == {"seq", "label"},
            f"CSV schema changed: {split}",
        )
        check(len(rows) == len(records), f"CSV/JSON row count mismatch: {split}")
        labels = [int(row["label"]) for row in rows]
        for csv_line, (row, label) in enumerate(zip(rows, labels), start=2):
            sequence_label_occurrences[row["seq"]].append(
                {"split": split, "csv_line": csv_line, "label": label}
            )
        check(
            all(label in (0, 1) for label in labels), f"Nonbinary region label: {split}"
        )
        counts = (len(rows), sum(labels), len(rows) - sum(labels))
        check(
            counts == EXPECTED[split], f"Frozen split counts changed: {split}: {counts}"
        )
        label_length_mismatches = non_c_positives = row_mismatches = positive_sites = 0
        for row, record in zip(rows, records):
            check(set(record) == {"seq", "label"}, f"JSON record schema changed: {split}")
            sequence = record["seq"]
            per_base = record["label"]
            check(set(sequence) <= set("ACGTN"), f"Unexpected sequence alphabet: {split}")
```





```
            check(
                all(
                    isinstance(value, (int, float))
                    and math.isfinite(value)
                    and value >= 0
                    for value in per_base
                ),
                f"Invalid per-base value: {split}",
            )
            label_length_mismatches += len(sequence) != len(per_base)
            non_c_positives += sum(
                value > 0 and base != "C" for base, value in zip(sequence, per_base)
            )
            positive_sites += sum(value > 0 for value in per_base)
            row_mismatches += row["seq"] != sequence or int(row["label"]) != int(
                any(value > 0 for value in per_base)
            )
        check(label_length_mismatches == 0, f"Per-base label length mismatch: {split}")
        check(non_c_positives == 0, f"Positive label at non-C base: {split}")
        check(row_mismatches == 0, f"CSV/JSON sequence or label disagreement: {split}")
        lengths = [len(row["seq"]) for row in rows]
        check(
            all(0 < length <= 1024 for length in lengths),
            f"Out-of-range region length: {split}",
        )
        regions[split] = collections.Counter(row["seq"] for row in rows)
        contexts[split] = candidate_contexts(records)
        report["datasets"][split] = {
            "region_rows": len(rows),
            "positive_regions": sum(labels),
            "zero_positive_regions": len(rows) - sum(labels),
            "minimum_length_nt": min(lengths),
            "maximum_length_nt": max(lengths),
            "full_1024nt_regions": lengths.count(1024),
            "unique_region_sequences": len(regions[split]),
            "cytidine_candidate_occurrences": sum(contexts[split].values()),
            "positive_site_occurrences": positive_sites,
            "csv_json_row_disagreements": row_mismatches,
            "label_length_mismatches": label_length_mismatches,
            "positive_labels_at_non_C": non_c_positives,
        }
    conflicting_sequences = [
        {
            "sequence_sha256": hashlib.sha256(sequence.encode("ascii")).hexdigest(),
            "length_nt": len(sequence),
            "occurrences": occurrences,
        }
        for sequence, occurrences in sequence_label_occurrences.items()
        if len({entry["label"] for entry in occurrences}) > 1
    ]
    report["identical_sequence_label_conflicts"] = {
        "unique_sequences": len(conflicting_sequences),
        "details": sorted(
            conflicting_sequences, key=lambda entry: entry["sequence_sha256"]
        ),
        "note": "Same sequence has both binary labels. No labels changed; locus/isoform provenance
.   is unavailable.",
    }
    report["cross_split_overlap"] = {}
    for left, right in itertools.combinations(SPLITS, 2):
        shared_regions = set(regions[left]) & set(regions[right])
```





```
        shared_contexts = set(contexts[left]) & set(contexts[right])
        report["cross_split_overlap"][f"{left}-{right}"] = {
            "shared_unique_region_sequences": len(shared_regions),
            "region_occurrences_left": sum(regions[left][seq] for seq in shared_regions),
            "region_occurrences_right": sum(
                regions[right][seq] for seq in shared_regions
            ),
            "shared_unique_101nt_C_centered_contexts": len(shared_contexts),
            "context_occurrences_left": sum(
                contexts[left][seq] for seq in shared_contexts
            ),
            "context_occurrences_right": sum(
                contexts[right][seq] for seq in shared_contexts
            ),
        }

    positives = read_csv(root / "data/PUF12_positive_with_strand.tsv", delimiter="\t")
    check(len(positives) == 12908, "Frozen positive TSV row count changed")
    coordinates = {
        (row["chromosome"], row["position"], row["strand"]) for row in positives
    }
    check(
        len(coordinates) == len(positives),
        "Duplicate positive TSV genomic coordinate/strand",
    )
    check(
        all(
            len(row["sequence_101nt"]) == 101 and row["sequence_101nt"][50] == "C"
            for row in positives
        ),
        "Positive TSV context length or central C mismatch",
    )
    alleles = collections.Counter(
        f'{row["strand"]}:{row["ref"]}>{row["alt"]}' for row in positives
    )
    check(
        dict(alleles) == {"-:G>A": 6588, "+:C>T": 6320},
        "Positive TSV strand/allele counts changed",
    )
    report["positive_tsv"] = {
        "rows": len(positives),
        "unique_coordinate_strand_tuples": len(coordinates),
        "strand_allele_counts": dict(sorted(alleles.items())),
        "all_contexts_101nt_centered_on_C": True,
        "note": "No raw supporting-read/depth columns or mapping to split rows; draft selection th
.
resholds cannot be reconstructed.",
    }

    prediction_path = root / RESULT_BASE / "test_evaluation/test_predictions.csv"
    predictions = read_csv(prediction_path)
    original_test = read_csv(root / "data/1024nt_dataset/orca_puf12_test.csv")
    check(len(predictions) == len(original_test), "Prediction/test row count mismatch")
    check(
        all(
            a["seq"] == b["seq"] and a["label"] == b["label"]
            for a, b in zip(predictions, original_test)
        ),
        "Prediction/test sequence or label alignment mismatch",
    )
    labels = [int(row["label"]) for row in predictions]
    classes = [int(row["predicted_class"]) for row in predictions]
```





```
    scores = [float(row["prob_class_1"]) for row in predictions]
    check(
        all(math.isfinite(score) and 0 <= score <= 1 for score in scores),
        "Invalid prediction probability",
    )
    check(
        all(
            math.isclose(float(row["prob_class_0"]) + score, 1.0, abs_tol=1e-9)
            for row, score in zip(predictions, scores)
        ),
        "Prediction probabilities do not sum to one",
    )
    check(
        all(
            predicted == int(float(row["prob_class_1"]) > float(row["prob_class_0"]))
            for row, predicted in zip(predictions, classes)
        ),
        "Saved class disagrees with probability argmax",
    )
    matrix = collections.Counter(zip(labels, classes))
    tn, fp, fn, tp = (matrix[(0, 0)], matrix[(0, 1)], matrix[(1, 0)], matrix[(1, 1)])
    metrics = {
        "accuracy": (tp + tn) / len(labels),
        "precision": tp / (tp + fp),
        "recall": tp / (tp + fn),
        "f1": 2 * tp / (2 * tp + fp + fn),
        "roc_auc": roc_auc(labels, scores),
        "pr_auc": average_precision(labels, scores),
        "true_negative": tn,
        "false_positive": fp,
        "false_negative": fn,
        "true_positive": tp,
        "num_samples": len(labels),
        "num_positive": sum(labels),
        "num_negative": len(labels) - sum(labels),
    }
    saved_metrics = read_json(root / RESULT_BASE / "test_evaluation/test_metrics.json")
    for key, value in metrics.items():
        check(
            math.isclose(value, saved_metrics[key], rel_tol=1e-10, abs_tol=1e-10),
            f"Saved metric mismatch: {key}",
        )
    report["stage_i_test_recomputed_from_saved_predictions"] = metrics
    report["stage_i_test_positive_prevalence"] = sum(labels) / len(labels)
    report["metric_definition"] = (
        "Historical pr_auc is non-interpolated average precision (AP), not trapezoidal PR area."
    )
    report["stage_i_best_search"] = read_json(root / RESULT_BASE / "best_trial.json")
    report["stage_i_final_validation"] = read_json(
        root / RESULT_BASE / "final_model/validation_metrics.json"
    )
    report["stage_i_final_hyperparameters"] = read_json(
        root / RESULT_BASE / "final_model/best_hyperparameters.json"
    )

    comparison = {}
    for path in sorted(
        (root / "results/model_1024_classifier").glob(
            "*/final_model/checkpoint-*/trainer_state.json"
        )
    ):
```





```
        state = read_json(path)
        evaluations = [entry for entry in state["log_history"] if "eval_pr_auc" in entry]
        comparison[path.parts[-4]] = {
            "source": path.relative_to(root).as_posix(),
            "evaluation_points_in_saved_checkpoint": len(evaluations),
            "best_logged_validation_AP": max(
                entry["eval_pr_auc"] for entry in evaluations
            ),
            "saved_checkpoint_step": state["global_step"],
        }
    report["stage_i_checkpoint_comparisons"] = comparison
    pretrain_path = (
        root
        / "results/model_continual_pretrain/transcriptome_pretrain/checkpoint-100000/trainer_state.json"
    )
    history = [
        entry
        for entry in read_json(pretrain_path)["log_history"]
        if "eval_masked_accuracy" in entry
    ]
    report["continued_pretraining"] = {
        "evaluation_points": len(history),
        "first_saved_evaluation": history[0],
        "last_saved_evaluation": history[-1],
        "original_model_0_505_baseline": "draft-only",
    }

    report["warnings"] = [
        "Exact region and C-centered context sequences occur across splits. Four unique region sequences have conflicting binary labels across their occurrences. Transcript/gene IDs and the split builder are absent; transcript-disjoint generalization cannot be verified.",
        "Stage II evaluates only regions with a labelled positive (150 validation, 140 test), excluding zero-positive regions. Draft ranking metrics are not an end-to-end gated pipeline evaluation.",
        "Per-candidate Stage II scores and machine-readable ranking evaluation results are absent. Draft ranking values cannot be independently recomputed from this bundle.",
        "Historical Stage I tokenization uses max_length=103 while training/evaluation declare 1026. The original tokenized cache is absent, so effective historical training context is unresolved.",
        "This publication repository omits large model weights and pretraining inputs that are retained in the original archive (see archive-inventory.json). The local LAMAR implementation, split-construction code, tokenized cache and pinned training environment are missing from the supplied archive. Historical training/evaluation is not runnable from this repository alone.",
        "Stage II training_state.json was invalid JSON (trailing comma); the original is preserved as training_state.json.txt rather than silently repaired.",
        "The drafts' original-model masked accuracy 0.505, transcriptome size, positive-label thresholds and Stage II AP around 0.2 are not independently recoverable from saved evaluation artifacts.",
        "Saved Stage I search and final-run settings differ from draft hyperparameters. Stage II source uses ranking loss, a scalar center-token head and fixed settings, not the draft binary-classifier/Optuna description.",
        "These checks establish artifact consistency, not model retraining, biological validation, probability calibration or off-target safety.",
    ]
    report["status"] = (
        "pass_with_documented_limitations" if not report["errors"] else "failed"
    )
    return report


```





```
def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--root", type=Path, default=Path(__file__).resolve().parents[1])
    parser.add_argument(
        "--output",
        type=Path,
        help="Write a new JSON report; refuses to overwrite an existing path",
    )
    args = parser.parse_args()
    try:
        report = audit(args.root.resolve())
    except (
        OSError,
        ValueError,
        KeyError,
        TypeError,
        IndexError,
        ZeroDivisionError,
    ) as error:
        report = {"status": "failed", "errors": [str(error)], "warnings": []}
    payload = json.dumps(report, indent=2, sort_keys=True, allow_nan=False) + "\n"
    if args.output:
        try:
            with args.output.open("x", encoding="utf-8") as handle:
                handle.write(payload)
        except OSError as error:
            print(f"Cannot create output: {error}", file=sys.stderr)
            return 2
    else:
        print(payload, end="")
    return 1 if report["errors"] else 0


if __name__ == "__main__":
    sys.exit(main())
```



